SecureScope

Automated Web Security Scanner

IBM Cybersecurity Internship — Project Report

Student name: _____

Institution: _____

Submission date: 26 October 2026

Project summary

SecureScope is a Python-based educational web-security auditing prototype. It reviews a limited set of publicly observable HTTP response settings, including HTTPS usage, selected security headers, and basic CORS response behavior. A Streamlit interface displays findings, severity labels, recommendations, and visual summaries. Users can export findings as HTML or CSV.

1. Introduction

Web security includes appropriate transport protection and response configuration. Manually reviewing headers can be repetitive, and technical output may be difficult for beginners to interpret. SecureScope organizes selected observations into a readable dashboard.

2. Problem statement

A beginner-friendly tool is needed to demonstrate automated checking of basic web security configuration and provide understandable remediation guidance.

3. Objectives

- Accept and validate a website URL.
- Check whether the final URL uses HTTPS.
- Inspect selected HTTP response headers.
- Observe a basic CORS response to a test Origin.
- Classify findings and show recommendations.
- Export results for documentation.

4. System design

Input: URL entered by the user. Processing: URL validation, HTTP request, response-header inspection, and simple rule-based scoring. Output: dashboard, findings table, recommendations, HTML report, and CSV report.

5. Tools and technologies

- Python — core logic.
- Requests — HTTP requests.
- Streamlit — interactive user interface.
- Pandas — tabular findings.
- Plotly — chart visualization.

6. Implementation

The application is separated into app.py for the interface and scanner.py for URL validation, HTTP requests, checks, scoring, and report generation. The application uses a short timeout and does not disable TLS certificate verification.

7. Checks included

- HTTPS usage after redirects.
- Strict-Transport-Security.
- Content-Security-Policy.
- X-Content-Type-Options.
- Referrer-Policy and Permissions-Policy.
- Clickjacking-related header indicators.
- Basic CORS response behavior.

- Server response header disclosure.

8. Testing plan

- Test a valid URL on an authorized training site.
- Check that malformed input produces a clear error.
- Check that HTTP URLs are flagged.
- Check that missing headers generate recommendations.
- Check timeout and TLS errors.
- Check that HTML and CSV downloads work.

9. Results

Complete this section after running the application. Insert genuine screenshots of the dashboard, findings table, chart, and downloaded report. Record the target only if you have authorization to assess it.

10. Limitations

The scanner checks only a small number of response settings. It does not crawl, authenticate, scan ports, exploit issues, or inspect server-side code. Missing headers are not automatically exploitable. A CORS observation alone does not prove impact. The risk score is an illustrative rule-based score, not a standard or certification.

11. Future enhancements

- Add unit tests for URL validation and scoring.
- Add configurable report branding.
- Add a controlled lab mode with mock responses.
- Add richer checks only in an authorized test environment.

12. Conclusion

SecureScope demonstrates the basic workflow of a defensive automated web scanner: request a URL, inspect selected response settings, explain findings, and produce a report. It is intended for learning and should be used only with explicit authorization.

References

Python: https://docs.python.org/3/ | Requests: https://requests.readthedocs.io/ | Streamlit: https://docs.streamlit.io/ | OWASP Secure Headers: https://owasp.org/www-project-secure-headers/

Responsible use: Scan only systems you own or have explicit permission to assess. This prototype is not a comprehensive penetration-testing product.